

Manuale tecnico e operativo Progetto Cave Report

Documentazione interna

23 giugno 2026

Indice

1	Scopo del programma	3
2	Struttura delle cartelle	3
3	Quale cartella copiare sulla macchina periodica	3
4	Comandi principali	4
4.1	Esecuzione normale in produzione	4
4.2	Esecuzione manuale dell'eseguibile	4
4.3	Test da sorgente Python	4
4.4	Scegliere un report specifico	4
4.5	Cambiare il numero di mesi	4
5	Funzionamento generale	4
6	File Python principali	5
6.1	main.py	5
6.2	report_config.py	5
6.3	df_format.py	5
6.4	dbf_to_df.py	6
6.5	df_compress.py	6
6.6	df_to_excel.py	6
7	Configurazione dei report	6
7.1	Differenza tra dataset e report	6
7.2	Filtri sulle righe del report	7
8	Come creare un nuovo report con lo stesso dataset	8
9	Come creare un nuovo dataset per DBF diversi	8
10	Regole importanti per il JSON	9
11	Campi calcolati disponibili	9
11.1	Younth	10
11.2	Cava	10
11.3	Val_tot	10

12 Crash e soluzioni	10
12.1 Errore: DBF mancanti	10
12.2 Errore: Config mancanti	10
12.3 Errore: Report non configurato	11
12.4 Errore: Dataset non configurato	11
12.5 Errore: Permission denied su Excel	11
12.6 Il report e' vuoto	11
12.7 I valori economici sono sbagliati	11
13 Log dell'esecuzione periodica	12
14 Attivita pianificata Windows	12
15 Rigenerare l'eseguibile	12
16 Procedura semplice per modificare un report	13
17 Procedura semplice per modificare i DBF usati	13
18 Checklist prima di mettere in produzione	13
19 Comando di test consigliato	14

1 Scopo del programma

Il programma genera report Excel partendo da file DBF di ARCA. In produzione i DBF vengono letti dalla cartella di rete:

```
\\172.16.2.13\arca\ditte\ICR
```

I file di configurazione, invece, stanno nella cartella **config** accanto al programma. Questo significa che sulla macchina periodica non bisogna creare una cartella **config** sulla rete: la configurazione viaggia insieme all'eseguibile.

Per impostazione predefinita il programma genera tre report separati:

- mese corrente;
- mese precedente;
- due mesi prima.

Esempio con data 23/06/2026:

```
report_2026-06.xlsx  
report_2026-05.xlsx  
report_2026-04.xlsx
```

2 Struttura delle cartelle

Percorso	Uso
script/	Codice Python sorgente. Serve per sviluppo e modifiche.
config/	Configurazioni dei report e mapping cave.
docs/	Documentazione tecnica e operativa del progetto.
export/	Excel generati. Non e' necessaria al funzionamento: puo' essere cancellata quando i file Excel non sono aperti.
release_fast/ProgettoCaveReport	Report consigliato per la macchina periodica. Contiene exe, config, batch e librerie interne.

3 Quale cartella copiare sulla macchina periodica

Usare la versione veloce:

```
release_fast\ProgettoCaveReport
```

La cartella deve essere copiata intera. Non copiare solo il file **ProgettoCaveReport.exe**, perche' servono anche **_internal**, **config** e i file batch.

Struttura attesa:

```
ProgettoCaveReport\  
  ProgettoCaveReport.exe  
  _internal\  
  config\  
    report_config.json  
    config_cava.txt  
  run_report_periodico.bat  
  installa_attivita_giornaliera.bat
```

I report verranno scritti in:

```
ProgettoCaveReport\export
```

I log verranno scritti in:

```
ProgettoCaveReport\logs
```

4 Comandi principali

4.1 Esecuzione normale in produzione

Dalla cartella del pacchetto:

```
run_report_periodico.bat
```

Questo batch lancia:

```
ProgettoCaveReport.exe --source rete --months 3 --no-open
```

4.2 Esecuzione manuale dell'eseguibile

```
ProgettoCaveReport.exe --source rete --months 3 --no-open
```

4.3 Test da sorgente Python

Sul computer di sviluppo, se si ha accesso alla rete:

```
python .\script\main.py --source rete --months 1 --no-open
```

L'opzione `-source esempi` esiste solo per test locali: funziona soltanto se viene creata manualmente una cartella `esempi` con dentro i DBF necessari.

4.4 Scegliere un report specifico

```
ProgettoCaveReport.exe --report righe_dettaglio --source rete --months 3 --no-open
```

4.5 Cambiare il numero di mesi

```
ProgettoCaveReport.exe --source rete --months 2 --no-open
```

5 Funzionamento generale

Il programma lavora in queste fasi:

1. legge `config/report_config.json`;
2. capisce quale report generare;
3. identifica il dataset da usare;
4. legge i DBF necessari;
5. filtra i documenti per periodo;
6. unisce le tabelle tramite i join configurati;
7. rinomina le colonne;

8. calcola eventuali campi derivati;
9. aggrega o lascia il dettaglio in base al report;
10. genera un file Excel formattato;
11. ripete l'esportazione per ogni mese richiesto.

Per i tre mesi il programma non rilegge i DBF tre volte: legge una sola volta l'intervallo complessivo, poi separa i dati per mese.

6 File Python principali

6.1 main.py

E' il punto di ingresso. Si occupa di:

- leggere gli argomenti da riga di comando;
- scegliere la sorgente DBF: rete in produzione, esempi solo per test locali;
- caricare la configurazione;
- controllare che DBF e config esistano;
- calcolare i mesi da elaborare;
- preparare i dati base;
- applicare la configurazione del report;
- chiamare l'export Excel.

Le costanti importanti sono:

```
CONFIG_DIR = BASE_DIR / "config"
RETE_DIR = Path(r"\\172.16.2.13\arca\ditte\ICR")
```

Se cambia il percorso di rete dei DBF, modificare `RETE_DIR`.

6.2 report_config.py

Legge `report_config.json`. Controlla che il report richiesto esista e aggancia al report anche il dataset collegato.

Errore tipico:

```
Report 'nome_report' non configurato
```

Significa che il nome passato con `-report` non esiste nel JSON.

6.3 df_format.py

Legge i DBF e fa i join. Non contiene piu' nomi fissi come `docrig` o `doctes`: segue quello che e' scritto nel dataset del JSON.

Se un nuovo report usa DBF diversi, normalmente si modifica il JSON, non questo file.

6.4 dbf_to_df.py

Contiene il lettore DBF veloce. Legge solo le colonne richieste dal config. Supporta i tipi DBF usati nel progetto: testo, date, numerici testuali, numerici binari FoxPro e booleani.

Se un DBF nuovo usa un tipo campo non gestito, il valore potrebbe uscire errato. In quel caso bisogna correggere la funzione:

```
_parse_valore(raw, tipo, decimali)
```

6.5 df_compress.py

Applica il raggruppamento. Se `group_by` e' vuoto, restituisce il dettaglio senza aggregare.

6.6 df_to_excel.py

Scrive e formatta l'Excel:

- formatta intestazioni e righe senza creare oggetti tabella Excel;
- applica larghezze colonne;
- formatta numeri;
- crea fogli riepilogo;
- gestisce il caso in cui il file sia aperto in Excel.

Se il file e' aperto, il programma crea una copia con data e ora invece di fallire.

7 Configurazione dei report

Il file principale e':

```
config\report_config.json
```

Ha due parti principali:

```
{
  "default_report": "cave_mensile",
  "datasets": {
    "...": {}
  },
  "reports": {
    "...": {}
  }
}
```

7.1 Differenza tra dataset e report

Dataset: dice da dove arrivano i dati.

Contiene:

- DBF da leggere;
- colonne da leggere;
- filtri;
- join;

- colonne da tenere;
- rinomina colonne;
- campi calcolati.

Report: dice come deve uscire l'Excel.

Contiene:

- dataset da usare;
- nome file;
- colonne finali;
- raggruppamenti;
- aggregazioni;
- formattazione;
- riepiloghi.

Nel report `cave_mensile` il raggruppamento include tipo documento, mese, gruppo doctrig, codice articolo, codice cliente/fornitore, ragione sociale, UM e descrizione riga. Questo evita di sommare insieme righe dello stesso articolo ma appartenenti a clienti o fornitori diversi, oppure con gruppo, unita' di misura o descrizione diversa. La ragione sociale resta quindi legata al relativo codice cliente/fornitore.

7.2 Filtri sulle righe del report

Ogni report puo' dichiarare `drop_rows`, cioe' regole per scartare righe prima del raggruppamento e prima dell'export Excel. Nel report attuale vengono scartate le righe senza codice articolo e con valore totale pari a zero:

```
"drop_rows": [
  {
    "all": [
      {
        "column": "Codice_Articolo",
        "operator": "blank"
      },
      {
        "column": "Val_tot",
        "operator": "eq",
        "value": 0
      }
    ]
  }
]
```

La regola usa `all`: la riga viene eliminata solo quando tutte le condizioni sono vere. Quindi una riga senza codice articolo ma con `Val_tot` diverso da zero resta nel report, perche' potrebbe contenere un valore economico da controllare.

8 Come creare un nuovo report con lo stesso dataset

Se i DBF sono gli stessi ma cambia solo l'Excel finale, basta aggiungere una nuova voce in **reports**.
Esempio:

```
"solo_cave": {
  "dataset": "arca_documenti",
  "output_file": "solo_cave_{periodo_file}.xlsx",
  "main_sheet": "Solo Cave",
  "freeze_panes": "A2",
  "group_by": ["Cava"],
  "aggregations": {
    "Quantita": "sum",
    "Val_tot": "sum"
  },
  "columns": [
    "Cava",
    "Quantita",
    "Val_tot"
  ],
  "number_formats": {
    "Quantita": "#,##0.00",
    "Val_tot": "#,##0.00"
  },
  "summaries": []
}
```

Poi si lancia:

```
ProgettoCaveReport.exe --report solo_cave --source rete --months 3 --no-open
```

9 Come creare un nuovo dataset per DBF diversi

Se un nuovo report usa DBF diversi o colonne diverse, bisogna aggiungere una nuova voce in **datasets**.

Schema semplificato:

```
"nuovo_dataset": {
  "sources": {
    "tabella_principale": {
      "file": "NOMEFILE.DBF",
      "suffix": "",
      "columns": ["ID", "DATA", "CODICE"],
      "date_filter": {
        "column": "DATA"
      }
    },
    "tabella_secondaria": {
      "file": "ALTROFILE.DBF",
      "suffix": "altro",
      "columns": ["IDTESTA", "DESCRIZION"],
      "filter_in": {
        "column": "IDTESTA",
        "source": "tabella_principale",
        "source_column": "ID"
      }
    }
  },
  "base_source": "tabella_principale",
}
```



```

"joins": [
  {
    "source": "tabella_secondaria",
    "left_on": "ID_",
    "right_on": "IDTESTA_altro",
    "how": "left"
  }
],
"keep_columns": [
  "DATA_",
  "CODICE_",
  "DESCRIZION_altro"
],
"rename_columns": {
  "DATA_": "data_documento",
  "CODICE_": "Codice",
  "DESCRIZION_altro": "Descrizione"
},
"calculated_fields": []
}

```

Poi il report lo usa così:

```

"mio_nuovo_report": {
  "dataset": "nuovo_dataset",
  "output_file": "mio_nuovo_report_{periodo_file}.xlsx",
  "main_sheet": "Report",
  "group_by": [],
  "columns": ["data_documento", "Codice", "Descrizione"],
  "summaries": []
}

```

10 Regole importanti per il JSON

- Le virgole sono obbligatorie tra elementi.
- Le stringhe vanno sempre tra doppi apici.
- L'ultima voce di una lista o oggetto non deve avere la virgola finale.
- Il nome usato in **reports** -> **dataset** deve esistere in **datasets**.
- I nomi colonne nei join devono essere quelli dopo il suffisso.

Esempio di suffisso:

```

columns: ["CODICE", "DESCRIZION"]
suffix: "anagrafe"

```

diventa:

```

CODICE_anagrafe
DESCRIZION_anagrafe

```

11 Campi calcolati disponibili

Nel dataset si possono indicare:

```

"calculated_fields": ["Younth", "Cava", "Val_tot"]

```

11.1 Younth

Trasforma la data documento nel mese del report, ad esempio:

```
06/2026
```

11.2 Cava

Usa i primi caratteri del codice articolo e li mappa tramite:

```
config\config_cava.txt
```

11.3 Val_tot

Calcola il valore totale della riga con la formula:

```
(prezzo_unitario - sconto_percentuale - sconto_valore + prezzo_trasporto) * quantita
```

12 Crash e soluzioni

12.1 Errore: DBF mancanti

Esempio:

```
ERRORE: DBF mancanti:  
\\172.16.2.13\arca\ditte\ICR\docrig.DBF
```

Cause possibili:

- la macchina non vede la rete;
- l'utente pianificato non ha permessi sulla share;
- il file DBF ha nome diverso;
- la cartella di rete e' cambiata.

Soluzioni:

1. Aprire Esplora File e provare a entrare in \\172.16.2.13\arca\ditte\ICR.
2. Controllare che l'attivit  pianificata usi un utente con accesso alla rete.
3. Controllare i nomi DBF in `report_config.json`.
4. Se il percorso rete cambia, modificare `RETE_DIR` in `script/main.py` e rigenerare l'eseguibile.

12.2 Errore: Config mancanti

Esempio:

```
ERRORE: Config mancanti:  
C:\...\ProgettoCaveReport\config\report_config.json
```

Significa che e' stata copiata solo l'exe. Copiare tutta la cartella:

```
release_fast\ProgettoCaveReport
```

12.3 Errore: Report non configurato

Esempio:

```
Report 'abc' non configurato
```

Il nome passato con `-report abc` non esiste in `report_config.json`.

12.4 Errore: Dataset non configurato

Il report punta a un dataset inesistente. Controllare:

```
"dataset": "nome_dataset"
```

e verificare che `nome_dataset` esista dentro `datasets`.

12.5 Errore: Permission denied su Excel

Se il file Excel e' aperto, Windows non permette di sovrascriverlo. Il programma crea una copia con timestamp, ad esempio:

```
report_2026-06_20260623_101500.xlsx
```

Soluzione: chiudere Excel prima del lancio periodico.

12.6 Il report e' vuoto

Possibili cause:

- nel mese richiesto non ci sono documenti;
- il filtro data punta alla colonna sbagliata;
- il join principale non trova righe;
- i DBF non sono stati ancora aggiornati.

Controllare nel JSON:

```
"date_filter": {  
  "column": "DATADOC"  
}
```

e il join tra testata e righe:

```
"left_on": "ID_",  
"right_on": "ID_TESTA_"
```

12.7 I valori economici sono sbagliati

Controllare i campi usati da `Val_tot`:

- `prezzo_unitario_docrig`
- `Sconti_Docrig`
- `sconti_valore_docrig`
- `Prezzo_Trasp`
- `Quantita`

Se i nomi cambiano nel config, bisogna aggiornare la funzione `_calcola_val_tot` in `main.py`.

13 Log dell'esecuzione periodica

Il batch scrive un file di log in:

```
logs\report_YYYYMMDD_HHMMSS.log
```

Dentro si trova:

- data e ora di avvio;
- cartella di lavoro;
- messaggi del programma;
- eventuale errore;
- codice uscita.

Codice uscita:

- 0: esecuzione riuscita;
- diverso da 0: errore.

14 Attività pianificata Windows

Per installare l'attività giornaliera:

```
installa_attivita_giornaliera.bat
```

Il batch crea una attività chiamata:

```
ProgettoCaveReport
```

Orario predefinito:

```
07:30
```

Per cambiarlo modificare:

```
set TASK_TIME=07:30
```

nel file:

```
installa_attivita_giornaliera.bat
```

15 Rigenerare l'eseguibile

Sul computer di sviluppo, dopo aver modificato Python o config, rigenerare la versione veloce:

```
python -m PyInstaller --noconfirm --clean --onedir --console --name ProgettoCaveReport --  
paths .\script --distpath .\release_fast --workpath .\build_pyinstaller_onedir .\  
script\main.py
```

Poi copiare config e batch dentro:

```
release_fast\ProgettoCaveReport
```

Il pacchetto finale da spostare e':

```
release_fast\ProgettoCaveReport
```

16 Procedura semplice per modificare un report

1. Aprire `config/report_config.json`.
2. Copiare un report esistente.
3. Cambiare il nome del report.
4. Cambiare `output_file`.
5. Cambiare `columns`.
6. Se serve aggregazione, cambiare `group_by` e `aggregations`.
7. Salvare il file.
8. Testare con:

```
python .\script\main.py --report nome_report --source rete --months 1 --no-open
```

17 Procedura semplice per modificare i DBF usati

1. Aprire `config/report_config.json`.
2. Aggiungere un dataset in `datasets`.
3. Definire ogni DBF in `sources`.
4. Scrivere le colonne necessarie in `columns`.
5. Definire `base_source`.
6. Definire i `join`.
7. Definire `keep_columns`.
8. Definire `rename_columns`.
9. Creare un report che usa quel dataset.
10. Testare sulla rete oppure su una cartella `esempi` creata manualmente.

18 Checklist prima di mettere in produzione

- La cartella `ProgettoCaveReport` contiene `ProgettoCaveReport.exe`, `_internal`, `config` e i `batch`.
- La macchina vede `\\172.16.2.13\arca\ditte\ICR`.
- L'utente dell'attività pianificata ha accesso alla rete.
- Il batch crea log in `logs`.
- La cartella `export` e' scrivibile.
- Nessun file Excel di output e' aperto durante l'esecuzione.
- Il primo lancio manuale e' stato provato.

19 Comando di test consigliato

Sul computer di sviluppo con accesso alla rete:

```
python .\script\main.py --source rete --months 1 --no-open
```

Sulla macchina di produzione:

```
ProgettoCaveReport.exe --source rete --months 1 --no-open
```

Se il test a un mese funziona, si puo' passare a:

```
ProgettoCaveReport.exe --source rete --months 3 --no-open
```